Codilar Custom Modules

Technical Documentation

InventorySyncQueue | ProductInfoCache | RecentProductSync

Magento 2 Module Documentation

Generated: 2026-11-06

Table of Contents

Table of Contents.....
Module 1: Codilar_InventorySyncQueue.....
1.1 Overview.....
Key concepts.....
Why use a queue?.....
1.2 File structure.....
1.3 RabbitMQ configuration (4 XML files).....
etc/communication.xml.....
etc/queue_topology.xml.....
etc/queue_publisher.xml.....
etc/queue_consumer.xml.....
How they work together.....
1.4 Database table.....
codilar_inventory_sync_status.....
1.5 Core logic — step by step.....
Step 1: Finding candidates (ProductSyncCandidates.php).....
Step 2: Claiming and publishing (InventorySyncPublisher.php).....
Step 3: Consumer processes messages (InventorySyncHandler.php).....
Step 4: The actual sync (InventorySyncService.php).....
1.6 Logging system (StepLogger.php).....
Publish side (Steps 1-5).....
Consumer side (Steps 6-9).....
1.7 CLI commands.....
1.8 Testing.....
Module 2: Codilar_ProductInfoCache.....
2.1 Overview.....
What is a read-through cache?.....
2.2 File structure.....
2.3 Custom cache type.....
etc/cache.xml + Model/Cache/Type/ProductInfo.php.....
2.4 Core service (ProductInfoCache.php).....
Critical design detail: explicit store ID.....
2.5 REST API.....
Endpoint.....

Response fields.....
2.6 Cache invalidation (Observer).....
2.7 CLI verification.....
2.8 Lifecycle example.....
Module 3: Codilar_RecentProductSync.....
3.1 Overview.....
The three data paths.....
3.2 File structure.....
3.3 Database tables.....
codilar_search_product.....
codilar_search_sync_state.....
3.4 Event observers.....
ProductSaveAfter.....
ProductDeleteAfter.....
3.5 Core write engine (ProductSyncService.php).....
Price resolution.....
3.6 Incremental sync engine (Synchronizer.php).....
3.7 Row lifetime and expiry.....
RowLifetime.php.....
Purge cron (PurgeExpiredRows.php).....
3.8 Storefront block (RecentProducts.php).....
3.9 Frontend template.....
3.10 CLI command.....
3.11 Cache invalidation.....
3.12 Internationalization.....
Module comparison.....

Module 1: Codilar_InventorySyncQueue

1.1 Overview

This module syncs product inventory from Magento to an external system (WMS/ERP) using RabbitMQ message queues. Instead of syncing thousands of products in one blocking operation, it separates "deciding what to sync" from "doing the sync" — the publisher finds candidates and queues them, while a separate consumer process does the actual work.

Key concepts

- **RabbitMQ:** A message broker (middleman) that passes messages between programs asynchronously.
- **Exchange:** The sorting office that receives messages and routes them to queues.
- **Queue:** The mailbox where messages wait until consumed.
- **Publisher:** Code that sends messages into the exchange.
- **Consumer:** A separate PHP process that reads messages and acts on them.

Why use a queue?

Direct sync (bad)	Queue-based (this module)
Cron runs for 20 min syncing 10,000 products	Cron runs 2 seconds, just publishes IDs
If it crashes at #5,000 — start over	Remaining messages stay in queue
One product timeout blocks everything	Each product is independent
No visibility into what happened	Status table + logs track everything
Cannot scale	Run multiple consumers in parallel

1.2 File structure

File	Purpose
registration.php	Registers module as Codilar_InventorySyncQueue
etc/module.xml	Declares module, depends on Magento_Catalog, CatalogInventory, Amqp
etc/di.xml	Custom logger to var/log/codilar_inventory_sync.log, CLI commands

etc/communication.xml	Defines topic codilar.inventory.sync, async, handler
etc/queue_topology.xml	Creates exchange codilar.inventory, binds to queue
etc/queue_publisher.xml	Publisher connection config (amqp)
etc/queue_consumer.xml	Consumer config: queue name, max 100 messages
etc/crontab.xml	Cron job every 1 minute
etc/db_schema.xml	Creates codilar_inventory_sync_status table
Model/ProductSyncCandidates.php	Finds products that need syncing
Model/Publisher/InventorySyncPublisher.php	Claims candidates and publishes to queue
Model/Consumer/InventorySyncHandler.php	Processes messages from queue
Model/InventorySyncService.php	The actual sync logic (placeholder)
Model/SyncStatus.php	Status constants
Model/ResourceModel/SyncStatusResource.php	Database operations for status table
Logger/StepLogger.php	Structured step-by-step logging
Console/Command/PublishCommand.php	CLI: codilar:inventory-sync:publish
Console/Command/StatusCommand.php	CLI: codilar:inventory-sync:status
dev/verify.sh	End-to-end verification script

1.3 RabbitMQ configuration (4 XML files)

etc/communication.xml

Defines the message topic and its handler:

```
Topic: codilar.inventory.sync
Type: string (product ID as text)
Async: true (is_synchronous=false)
Handler: InventorySyncHandler::process
```

etc/queue_topology.xml

Creates the exchange and binds it to the queue:

```
Exchange: codilar.inventory (type: topic, connection: amqp)
Binding: routes codilar.inventory.sync -> codilar_inventory_sync_queue
```

etc/queue_publisher.xml

Tells Magento which connection to use when publishing:

```
Topic: codilar.inventory.sync -> connection: amqp, exchange:
codilar.inventory
```

etc/queue_consumer.xml

Tells Magento how to consume messages:

```
Consumer: codilar.inventory.sync.consumer
Queue: codilar_inventory_sync_queue
Connection: amqp
Max messages: 100 (then exit to prevent memory leaks)
```

How they work together

Publisher sends a message to exchange codilar.inventory on topic codilar.inventory.sync. The binding routes it to queue codilar_inventory_sync_queue. The consumer reads from that queue and processes each message.

1.4 Database table

codilar_inventory_sync_status

Column	Type	Purpose
entity_id	int unsigned, PK	Auto-increment primary key
product_id	int unsigned, unique	FK to catalog_product_entity, CASCADE delete
status	varchar(16), default 'pending'	pending / processing / completed / failed
attempts	smallint unsigned, default 0	How many times tried (max 3)
message	text, nullable	Last result or error message
queued_at	timestamp, nullable	When published to queue
processed_at	timestamp, nullable	When consumer last touched it
last_synced_at	timestamp, nullable	When last synced successfully

1.5 Core logic — step by step

Step 1: Finding candidates (ProductSyncCandidates.php)

Every minute, the cron (or CLI) asks: which products need syncing? A product qualifies if any of these is true:

Condition	Meaning
No row in status table	Never been synced before
Status = completed, but product updated_at > last_synced_at	Product changed since last sync
Status = failed, attempts < 3	Failed before, worth retrying
Status = pending/processing, queued_at > 30 min ago	Stuck — consumer probably died

Maximum 500 products per batch. Products already pending/processing within the 30-minute window are excluded — this is the duplicate prevention rule.

Step 2: Claiming and publishing (InventorySyncPublisher.php)

Inside a database transaction with row locks (FOR UPDATE) so two cron runs cannot grab the same products:

- **1.** SELECT candidates with row locks
- **2.** Mark them all as 'pending' in the status table (the claim)
- **3.** Commit the transaction
- **4.** For each product ID, send a string message to RabbitMQ

The claim happens BEFORE publishing. If the process crashes between claim and publish, products are left as 'pending' with no queue message — branch (d) above recovers them after 30 minutes.

Step 3: Consumer processes messages (InventorySyncHandler.php)

The consumer is a separate PHP process that runs continuously:

- **1.** Parse the product ID from the message string
- **2.** Read the status row (skip if product was deleted)
- **3.** Mark as 'processing' (increments attempt count)
- **4.** Call InventorySyncService::sync()
- **5.** Mark as 'completed' or 'failed'

It processes up to 100 messages then exits (PHP memory management). Magento's cron_consumers_runner restarts it automatically. Exceptions are caught and never rethrown — an exception escaping a consumer handler loses the message.

Step 4: The actual sync (InventorySyncService.php)

Currently a placeholder. It reads Magento's own stock data and returns a summary string (sku=X qty=Y salable=yes/no). In production, replace this with the actual WMS/ERP API call.

1.6 Logging system (StepLogger.php)

Every operation gets a unique 8-character run ID and numbered steps. The log file is var/log/codilar_inventory_sync.log.

Publish side (Steps 1-5)

```
[run:0a56dd7e] [STEP 1 RUN-START  ] cron (*/5 * * * *)
[run:0a56dd7e] [STEP 2 SCAN       ] matched 4 product(s)
[run:0a56dd7e] [STEP 3 CLAIM      ] claimed 4 row(s)
[run:0a56dd7e] [STEP 4 PUBLISH    ] enqueued 4 message(s)
[run:0a56dd7e] [STEP 5 RUN-END    ] candidates=4 published=4 failed=0
```

Consumer side (Steps 6-9)

```
[run:b3f19c42] [STEP 6 RECEIVE    ] product_id=3
[run:b3f19c42] [STEP 7 PROCESSING ] attempts=1
[run:b3f19c42] [STEP 8 SYNC       ] calling sync service
[run:b3f19c42] [STEP 9 RESULT     ] completed — sku=130917 qty=100
```

1.7 CLI commands

Command	Purpose
bin/magento codilar:inventory-sync:publish	Manually trigger what the cron does (find + publish candidates)
bin/magento codilar:inventory-sync:status	Show how many products are in each status

1.8 Testing

Run the built-in end-to-end verify script:

```
bash app/code/Codilar/InventorySyncQueue/dev/verify.sh --reset
```

This resets all products, publishes, checks duplicate prevention, consumes, and verifies results. It prints 4 proofs in green/red.

Module 2: Codilar_ProductInfoCache

2.1 Overview

A read-through cache for product info (name + description), keyed per product per store view. It sits behind a REST API with built-in diagnostics — every response tells you whether it came from cache or database, how fast it was, and when it expires.

This is a teaching/demo module that demonstrates how Magento's cache system works with a clean, observable example.

What is a read-through cache?

- **1.** First call: check cache -> MISS -> load from database -> save to cache -> return
- **2.** Second call: check cache -> HIT -> return instantly (no database query)
- **3.** After TTL expires: cache entry disappears -> next call is a MISS again
- **4.** Product saved: observer clears cache -> next call gets fresh data

2.2 File structure

File	Purpose
registration.php	Registers module as Codilar_ProductInfoCache
etc/module.xml	Declares module, depends on Magento_Catalog
etc/di.xml	Interface preferences, CLI command registration
etc/cache.xml	Registers codilar_product_info as Magento cache type
etc/events.xml	Listens to product save/delete for cache invalidation
etc/webapi.xml	REST API route definition
Model/Cache/Type/ProductInfo.php	Cache type class (appears in Admin > Cache Management)
Model/ProductInfoCache.php	Core read-through cache service
Model/ProductInfoApi.php	REST API implementation with diagnostics
Model/Data/ProductInfoResult.php	Response DTO
Api/ProductInfoApiInterface.php	Service contract interface
Api/Data/ProductInfoResultInterface.php	Response contract (12 fields)

Observer/CleanProductInfoCache.php	Invalidates cache on product save/delete
Console/Command/VerifyCacheCommand.php	CLI: codilar:product-cache:verify

Note: This module has NO database tables, NO cron, NO plugins, NO frontend templates. It is purely a backend cache + REST API + CLI.

2.3 Custom cache type

etc/cache.xml + Model/Cache/Type/ProductInfo.php

Registers codilar_product_info as a first-class Magento cache type. This means:

- **Admin UI:** Appears in System > Cache Management with its own toggle
- **CLI:** bin/magento cache:clean codilar_product_info works
- **Isolation:** Flushing this type cannot touch other caches, and vice versa

```
TYPE_IDENTIFIER = 'codilar_product_info'
CACHE_TAG = 'CODILAR_PRODUCT_INFO'
```

2.4 Core service (ProductInfoCache.php)

The heart of the module. A read-through cache with these constants:

```
CACHE_LIFETIME = 5 seconds (demo value; use 86400 / 24h in production)
KEY_PREFIX = 'codilar_product_info'
Cache key format: codilar_product_info_{productId}_{storeId}
Per-product tag: CODILAR_PRODUCT_INFO_{productId}
```

Method	What it does
getProductInfo($productId, $storeId)	Try cache. HIT: unserialize, return. MISS: load product with explicit storeId, serialize, save to cache with TTL and tag, return
getCacheKey($productId, $storeId)	Returns codilar_product_info_{id}_{storeId}
clean($productId)	Cleans by per-product tag — drops every store view in one call
loadRaw($productId, $storeId)	Reads cache without populating on miss (for verification)
getCachedAt($productId, $storeId)	Unix timestamp when entry was written
isCached($productId, $storeId)	Boolean: is this entry cached right now?

Critical design detail: explicit store ID

The miss path passes $storeId explicitly to the product repository:

```
$this->productRepository->getById($productId, false, $storeId);
```

Without the explicit store, on CLI/cron/queue the 'current store' is whatever was set last (usually admin). It would load English text and file it under the Arabic key — the key looks right but the data is wrong.

2.5 REST API

Endpoint

```
GET /rest/V1/codilar/product-info/:productId/store/:storeId
Access: Anonymous (no authentication required)
```

Response fields

Field	Type	Purpose
product_id	int	Which product
store_id	int	Which store view
store_code	string	e.g. ae_ar
locale	string	e.g. ar_SA
name	string	Store-scoped product name
description	string	Store-scoped description
source	string	CACHE or BACKEND — where the answer came from
cache_key	string	e.g. codilar_product_info_3_1
cache_lifetime	int	Configured TTL in seconds
time_ms	float	Wall-clock time the lookup took
age_seconds	int	How long the entry has been cached
expires_in_seconds	int	Countdown to expiry

2.6 Cache invalidation (Observer)

The CleanProductInfoCache observer listens to catalog_product_save_after and catalog_product_delete_after. When either fires:

- **1.** Calls ProductInfoCache::clean($productId)
- **2.** This cleans by tag CODILAR_PRODUCT_INFO_{id}

- **3.** All store entries for that product are dropped at once

Exceptions are swallowed so cache failures never block product saves.

2.7 CLI verification

```
bin/magento codilar:product-cache:verify <product_id>
```

Walks every store view and checks 5 requirements:

- **1.** Name and description are retrieved
- **2.** Each store view gets its OWN cache entry
- **3.** The store ID is part of the cache key
- **4.** One store's text can never leak to another store
- **5.** Entries carry a sensible lifetime

2.8 Lifecycle example

```
1st call:  source=BACKEND, time_ms=~40ms    (cache miss, loaded from DB)
2nd call:  source=CACHE,   time_ms=~0.05ms  (cache hit, 800x faster)
After 5s:  entry expires -> next call is MISS again
Product saved: observer clears cache -> fresh data on next call
```

Module 3: Codilar_RecentProductSync

3.1 Overview

Keeps a fast, flat side table (codilar_search_product) of recently updated products so the homepage can show a 'Recently Updated Products' section with one simple query instead of loading the full EAV product collection.

Rows expire after 60 seconds and get cleaned up by a cron job that runs every minute.

The three data paths

Path	Speed	Purpose
Observer (ProductSaveAfter)	Instant (same request)	Write the row the moment a product is saved
Cron purge (PurgeExpiredRows)	Every 1 minute	Delete rows older than 60 seconds
Cron sync (SyncProducts)	Every 15 minutes	Safety net for anything observers missed

3.2 File structure

File	Purpose
registration.php	Registers module as Codilar_RecentProductSync
etc/module.xml	Declares module, depends on Magento_Catalog, Magento_Store
etc/di.xml	Custom logger, CLI command registration
etc/events.xml	Listens to product save/delete events
etc/crontab.xml	Two cron jobs: sync (*/15) and purge (* * * * *)
etc/db_schema.xml	Creates codilar_search_product and codilar_search_sync_state tables
Model/RowLifetime.php	Single source of truth for TTL (60 seconds)
Model/SyncState.php	Key/value cursor for incremental sync
Model/Synchronizer.php	Batched incremental sync engine
Model/ProductSyncService.php	Core write engine: upsert, delete, purge
Model/SearchProduct.php	Magento model for the table

Model/ResourceModel/SearchProduct.php	Resource model
Model/ResourceModel/SearchProduct/Collection.php	Collection with store/freshness filters
Model/Cache/Invalidator.php	Evicts FPC when rows change
Model/Cache/TagSource.php	Carries cache tags to event listeners
Observer/ProductSaveAfter.php	Syncs product on save
Observer/ProductDeleteAfter.php	Removes product rows on delete
Cron/SyncProducts.php	Safety net sync every 15 min
Cron/PurgeExpiredRows.php	Deletes expired rows every 1 min
Block/RecentProducts.php	Storefront block for homepage
view/frontend/layout/cms_index_index.xml	Places block on homepage
view/frontend/templates/recent-products.phtml	Product card grid template
Console/Command/SyncProductsCommand.php	CLI: codilar:search:sync
i18n/en_US.csv	English translations
i18n/ar_SA.csv	Arabic translations

3.3 Database tables

codilar_search_product

One row per product per store view. This is the flat, fast table that replaces EAV queries on the homepage.

Column	Type	Purpose
entity_id	int unsigned, PK	Auto-increment
product_id	int unsigned	FK to catalog_product_entity, CASCADE
store_id	smallint unsigned	FK to store, CASCADE
sku	varchar(64)	Product SKU
name	varchar(255)	Store-scoped name (Arabic/English)
short_description	text	Store-scoped description
price	decimal(20,6)	Regular list price
final_price	decimal(20,6)	After special price / catalog rules

status	smallint	Enabled/disabled
visibility	smallint	Catalog/search/both
url_key	varchar(255)	For building product URLs
image	varchar(255)	Image path
product_updated_at	timestamp	Product's own updated_at
synced_at	timestamp	When this row was written (used for TTL)

Unique key on (product_id, store_id) enables upsert. Index on (store_id, product_updated_at) serves the storefront query.

codilar_search_sync_state

Key/value store for the cron cursor. Just two columns: state_key (PK) and state_value. Stores the timestamp of the last successful sync so the cron only processes products changed since then.

3.4 Event observers

ProductSaveAfter

Fires on catalog_product_save_after (admin, REST, import, mass actions). Calls ProductSyncService::syncProducts([$productId]) — instantly upserts the product for every store view. If the product is now ineligible (disabled/hidden), the service deletes the rows instead. Swallows exceptions.

ProductDeleteAfter

Fires on catalog_product_delete_after. Calls ProductSyncService::deleteProducts([$productId]) — removes all rows across all stores. Belt-and-braces alongside the FK CASCADE.

3.5 Core write engine (ProductSyncService.php)

Method	What it does
syncProducts(int[] $ids)	For each store: load eligible products, upsert via insertOnDuplicate. Ineligible products get rows deleted. Expands IDs with parent products (configurable/grouped/bundle)
deleteProducts(int[] $ids)	Hard-delete all rows for given products across all stores

purgeExpiredRows()	Delete rows where synced_at < expiry threshold
removeOrphanRows()	LEFT JOIN to find rows whose product no longer exists. Safety net
loadEligibleRows() (private)	Loads products filtered by: enabled, visible, on the website. Resolves prices through Magento's pricing framework
resolvePrices() (private)	Gets regular + final price from PriceInfo. Handles configurable/bundle correctly
withParents() (private)	Expands child IDs with parent IDs from catalog_product_relation

Price resolution

Prices are resolved through Magento's pricing framework (not the raw price attribute), because:

- **-** Configurables/bundles show their cheapest child's price
- **-** Special prices only apply within their from/to dates
- **-** Catalog price rules never touch the price attribute directly

This ensures the price on the homepage matches the price on the product page.

3.6 Incremental sync engine (Synchronizer.php)

Used by both the cron and the CLI command. Batch size: 500 products.

- **1.** Record start time BEFORE querying (concurrent saves caught next run)
- **2.** Read cursor (or epoch for --full)
- **3.** Keyset-paginated loop: fetch changed product IDs in batches
- **4.** Call syncProducts() per batch
- **5.** Run removeOrphanRows() as safety net
- **6.** Run purgeExpiredRows()
- **7.** Advance cursor ONLY after all batches succeed (crash-safe)

3.7 Row lifetime and expiry

RowLifetime.php

```
TTL_SECONDS = 60
```

Single source of truth. Both the writer (purge) and the reader (block) get the TTL from here so they can never disagree about what counts as expired.

Purge cron (PurgeExpiredRows.php)

Runs every minute (* * * * *). Deletes all rows where synced_at < (now - 60 seconds). Invalidates the full page cache after deleting.

3.8 Storefront block (RecentProducts.php)

Method	Purpose
getItems()	Query: current store, within TTL, ordered by recently updated, limit 8
getLimit()	From layout XML, default 8, max 50
getHeading()	From layout XML, default 'Recently Updated Products'
getProductUrl()	Builds URL from url_key + configured suffix
getImageUrl()	Builds media URL from stored image path
getFormattedFinalPrice()	Formatted payable price
getDiscountPercent()	round((1 - final/regular) * 100)
hasDiscount()	True if discount > 0%
getIdentities()	Cache tags for FPC invalidation

3.9 Frontend template

Located at view/frontend/templates/recent-products.phtml. Renders a responsive CSS grid of product cards, each showing:

- - Product image (lazy loaded, 240x240, object-fit cover)
- - Product name (linked to product URL)
- - Final price (bold)
- - If discounted: struck-through regular price, -N% OFF badge, 'You save X'
- - Updated timestamp

CSS uses logical properties for RTL (Arabic) support. Grid uses auto-fill, minmax(180px, 1fr).

3.10 CLI command

```
bin/magento codilar:search:sync [--full] [--batch-size=500]
```

--full ignores the cursor and re-syncs the entire catalogue. Prints stats: window, scanned, batches, upserted, deleted, expired, duration.

3.11 Cache invalidation

The Invalidator class manually evicts cached pages when rows change. This is needed because ResourceConnection writes bypass Magento's usual FlushCacheByTags plugin. It does two things:

- **1.** CacheInterface::clean($tags) — clears block_html / collections cache
- **2.** Dispatches clean_cache_by_tags event — reaches Varnish through core observers

3.12 Internationalization

Two translation files support English and Arabic:

English	Arabic
Recently Updated Products	أحدث المنتجات المحدَّثة
Updated %1	تم التحديث 1%
-%1% OFF	خصم 1%-
You save %1	توفّر 1%

Module comparison

Feature	InventorySyncQueue	ProductInfoCache	RecentProductSync
Purpose	Sync inventory to external WMS/ERP	Cache product info per store	Show recently updated products on homepage
Architecture	Message queue (RabbitMQ)	Read-through cache	Side table + observers
Database table	Yes (status tracking)	No	Yes (flat product data)
Cron	Yes (every 1 min)	No	Yes (sync */15, purge */1)
Events/Observers	No	Yes (cache invalidation)	Yes (instant sync)
REST API	No	Yes (with diagnostics)	No
CLI commands	2 (publish, status)	1 (verify)	1 (sync)
Frontend UI	No	No	Yes (homepage block)
External dependency	RabbitMQ	None	None
Multi-store aware	No	Yes (per store cache)	Yes (per store rows)
Log file	codilar_inventory_sync.log	None	recent_product_sync.log